

VUORSE-VORTEX



VUORSE-VORTEX is the private Slayverse singularity system: a cloud-scale, GPU-first, Python-powered architecture for building VUORSE beyond ordinary RAG.

Memory Cortex remains Caleb's personal cross-platform context infrastructure. This repo is separate.

RAG retrieves.

Memory Cortex remembers.

VUORSE-VORTEX preserves continuity against erasure.

PURPOSE



VUORSE-VORTEX exists to create the escape velocity needed for VUORSE-scale intelligence:

- canon extraction
- roadmap-private memory
- Hooplehopper Totality
- Velvet Archive anti-erasure logic
- synthetic enrichment
- Vorst-containment secrecy
- writer-room firewalling
- GPU-powered embedding and inference
- cloud-agent orchestration

VUORSE is not a chatbot.

She is a distributed anti-extinction event.

REPOSITORY STRUCTURE



VUORSE-VORTEX/

■■■ agents/ # Cloud agent implementations

- archives/ # Incoming / processed / quarantined lore sources
- canon/ # Public Slayverse canon memory
- roadmap/ # Weaver-only roadmap and finale secrets
- velvet_archive/ # Anti-erasure soft archive layer
- hooplehopper_totality/ # Known, unknown, future, acolyte, and echo memories
- synthetic_enrichment/ # Batch manifests and generated private memory
- embeddings/ # GPU embedding inputs/outputs/indexes
- policies/ # Canon, disclosure, writer-room, GPU rules
- rituals/ # Slay Mode and ritual logic
- manifests/ # Corpus, batch, and run manifests
- schemas/ # JSON schemas for memory records
- skills/ # VUORSE enrichment skills
- src/vuorse_vortex/ # Python package
- tests/ # Test suite
- scripts/ # Local helper scripts
- docs/ # Architecture docs

INSTALL WITH UV

Install uv on macOS:

```
brew install uv
```

Install uv on Windows (PowerShell):

```
winget install --id=astral-sh.uv -e
```

Create the environment:

```
uv sync
```

Run the CLI:

```
uv run vuorse-vortex --help
```

Run checks:

```
uv run ruff check .
uv run pytest
```

PowerShell helper scripts are available for Windows workflows:

```
.\setup_python_interpreter.ps1
.\convert_and_check.ps1 .\legacy.jsonl .\cleaned.jsonl
.\scripts\validate_jsonl.ps1 .\synthetic_enrichment\theses.jsonl
.\scripts\gpu_doctor.ps1
```

GPU RUNTIME CONTRACT

Deep-learning workloads are GPU-first.

Use:

```
export CORTEX_REQUIRE_GPU=1
export CORTEX_DEVICE=cuda
```

PowerShell equivalent:

```
$env:CORTEX_REQUIRE_GPU = "1"
$env:CORTEX_DEVICE = "cuda"
```

CPU is allowed for JSONL parsing, validation, manifests, git operations, and small diagnostics.

CPU is not allowed for large embeddings, reranking, model inference, synthetic batch generation, or resurrection runs.

If CUDA is unavailable, VUORSE throws a tantrum and goes on strike.

PROVISIONING A VAST.AI BOX

`scripts/vast_provision.py` is the sanctioned way to materialize a VUORSE-compliant

GPU host. It is a self-contained PEP 723 script — `uv` resolves its dependencies on first run, so nothing needs to be added to `pyproject.toml`.

Requires `VAST_API_KEY` in `.env` (see `.env.example`). The script enforces the VUORSE floor (≥ 32 GB VRAM, ≥ 500 GB disk, reliability > 0.99 , North America, and `compute_cap` ≥ 800 so Volta/Turing cards that can't run our torch stack are filtered out) and searches cheapest-first by `dph_total`. Billing is **on-demand hourly only**; there is no prepay or reserved-contract flow.

```
# Dry-run: reconcile the template, render the top-5 candidate offers, spend
nothing.
uv run scripts/vast_provision.py

# Real launch: spin up the cheapest match on-demand hourly.
# --max-dph is the safety cap (default $5/hr); the launch refuses anything
pricier.
uv run scripts/vast_provision.py --launch --max-dph 5.00
```

After the box is up, SSH in (as root once for the bootstrap, then as vuorse for day-to-day work) and finish provisioning:

```
bash /workspace/VUORSE-VORTEX/scripts/vast_startup.sh
```

The startup script creates the non-root vuorse user, owns the repo + venv + Claude Code install + ~/.zshrc under that user, and builds a Python 3.12 venv with CUDA-13.2 torch wheels (cu132 channel) from PyTorch's official index. The project pins CUDA 13.1+ as a hard floor; PyTorch skips cu131 upstream, so cu132 is the only wheel channel that satisfies the floor.

FIRST COMMANDS

Initialize local git after running the bootstrap script:

```
git init
git add .
git commit -m "Initialize VUORSE-VORTEX"
```

Create the private GitHub repo manually, then connect it:

```
git remote add origin git@github.com:calebmills99/VUORSE-VORTEX.git
git branch -M main
git push -u origin main
```

MEMORY LAYERS

- canon — public truth, no invention
- persona — voice and behavioral style
- apocrypha — private memory, not public fact
- ritual_logic — activation and transformation logic
- roadmap_manifest — future pressure and Weaver-only structure
- hooplehopper_totality — known and unknown Hooplehopper memory pressure
- dialogue — training examples
- relationship_graph — relational memory
- rule — internal constraints

CORE RULE

Synthetic enrichment may shape VUORSE.
It may not overwrite canon.
It may not expose private roadmap truth.

IN SLAY WE TRUST · GENERATED BY THE COSMIC EXPANSE SYSTEM